

Morphic Studio Software Architecture Specification

Document 45 — DevOps, Infrastructure & Deployment Architecture

Version: 1.0

Status: Approved

1. Purpose

This document defines the DevOps, infrastructure, deployment, monitoring, and operational architecture for Morphic Studio.

Its purpose is to ensure that development, testing, staging, and production environments remain reproducible, secure, scalable, observable, and resilient throughout the product lifecycle.

2. DevOps Philosophy

The operational architecture follows these principles:

2.1 Infrastructure as Code

All infrastructure must be defined in version-controlled code.

Manual production configuration should be avoided whenever possible.

Infrastructure should be reproducible from source control.

2.2 Automation First

Routine operational tasks should be automated.

Examples include:

- Testing
 - Building
 - Deployment
 - Security scanning
 - Database migrations
 - Backup scheduling
 - Health verification
-

2.3 Immutable Deployments

Every deployment should be generated from a versioned build artifact.

Production servers should not be modified manually after deployment.

2.4 Environment Consistency

Local development, testing, staging, and production should behave as consistently as practical.

Differences should be limited to configuration and scale.

2.5 Continuous Improvement

Operational metrics should continuously guide reliability, performance, and cost optimization.

3. Environment Strategy

The platform supports:

Development

Local developer environment.

Testing

Automated testing environment.

Staging

Production-like validation.

Production

Customer-facing environment.

Future:

Preview environments for pull requests.

4. Source Control Workflow

Use Git with protected branches.

Recommended branches:

- main
- develop
- release/*
- hotfix/*
- feature/*

Changes are introduced through pull requests with automated validation.

5. Continuous Integration (CI)

Every commit triggers automated workflows.

CI pipeline includes:

- Dependency installation
- Static analysis
- Linting
- Type checking
- Unit tests
- Integration tests
- Accessibility checks
- Security scanning
- Build verification

Build failures prevent merging.

6. Continuous Delivery (CD)

Successful builds are automatically deployable.

Deployment stages:

Development

↓

Testing

↓

Staging

↓

Production

Production deployments require approval policies appropriate for the release process.

7. Container Strategy

Applications should run in containers.

Container images must be:

- Versioned
- Reproducible
- Minimal
- Secure
- Scanned for vulnerabilities

Development containers should closely mirror production.

8. Configuration Management

Configuration is externalized.

Examples:

- Environment variables
- Secret management
- Feature flags
- Deployment configuration

Configuration should never require source code modification.

9. Secret Management

Secrets include:

- Database credentials
- AI provider keys
- OAuth credentials
- Encryption keys
- Webhook secrets
- Storage credentials

Secrets must never be committed to source control.

Support secure rotation and auditing.

10. Monitoring & Observability

Monitor:

- Application health
- API latency
- Database performance
- Queue health
- AI provider availability
- Storage utilization
- CPU
- Memory
- Network
- Deployment health

Centralized dashboards should provide operational visibility.

11. Logging Strategy

Implement structured logging.

Every log entry should include:

- Timestamp
- Severity
- Service
- Correlation ID
- Request ID
- User ID (when appropriate)
- Environment

Sensitive information must never appear in logs.

12. Alerting

Generate alerts for:

- Service failures
- High error rates
- Database issues
- Queue backlogs
- Failed deployments
- Security incidents
- AI provider outages
- Storage failures

Alerts should include sufficient diagnostic information.

13. Backup & Disaster Recovery

Support:

- Automated backups
- Encrypted backups
- Point-in-time recovery
- Disaster recovery procedures
- Recovery testing
- Backup validation

Recovery objectives should be documented and periodically verified.

14. Scaling Strategy

Support independent scaling of:

- Frontend
- Backend
- AI Gateway
- Background workers
- Search
- Database replicas
- Object storage
- Rendering services

Horizontal scaling is preferred where practical.

15. Release Management

Every release includes:

- Version number
- Changelog
- Migration notes
- Rollback plan
- Deployment verification
- Post-release monitoring

Releases should be repeatable and traceable.

16. Cost Management

Monitor:

- Compute usage
- Storage costs
- AI provider costs
- Bandwidth
- Database utilization
- Background worker utilization

Operational decisions should balance performance and cost.

17. Business Continuity

Prepare for:

- Cloud provider outages
- AI provider failures
- Database failures
- Storage failures
- Network disruptions
- Human error

Document recovery procedures for critical services.

18. Acceptance Criteria

The DevOps architecture is successful if:

- Infrastructure is reproducible.
 - Deployments are automated.
 - Monitoring provides actionable insights.
 - Secrets remain protected.
 - Recovery procedures are tested.
 - Scaling can occur without redesign.
 - Releases are reliable and reversible.
 - Operational costs remain measurable and manageable.
-

Conclusion

The Morphe Studio DevOps architecture establishes a modern operational foundation built on automation, Infrastructure as Code, observability, security, and resilience. By standardizing environments, deployment pipelines, monitoring, and disaster recovery, the platform is prepared for reliable operation from local development through global production deployments.